



Avaliação

JavaScript (Temas 2 a 6) + Ferramentas para Desenvolvimento Web

Questão 1 – Verdadeiro (V) ou Falso (F) – 0,4

Ferramentas

Analise as afirmações sobre **Ferramentas para Desenvolvimento Web**:

- I. O Visual Studio Code (VS Code) é um editor de código gratuito que suporta extensões como Live Server e Live Preview.
- II. O GitHub Pages é um serviço pago para hospedar sites estáticos com suporte a repositórios privados.
- III. O OneCompiler executa código JavaScript em servidores remotos usando Node.js, e não diretamente no navegador.

Assinale a alternativa correta:

- ☐ a) V – V – F
- ☒ b) V – F – V
- ☐ c) F – V – V
- ☐ d) V – V – V
- ☐ e) F – F – V

Questão 2 – Verdadeiro (V) ou Falso (F) – 0,4

JavaScript

Analise as afirmações sobre **Variáveis em JavaScript**:

- I. A palavra-chave `const` declara uma variável que não pode ser reatribuída.
- II. A palavra-chave `let` tem escopo de bloco e permite reatribuição.
- III. A palavra-chave `var` tem escopo de bloco, assim como `let`.

Assinale a alternativa correta:

- ☒ a) V – V – F
- ☐ b) V – F – V
- ☐ c) F – V – V
- ☐ d) V – V – V
- ☐ e) F – F – V

Questão 3 – Múltipla Escolha – 0,35

Ferramentas

Sobre a política de segurança **CORS** (Cross-Origin Resource Sharing) e a leitura de arquivos `.json`, assinale a alternativa **CORRETA**:

- ☐ a) Arquivos JSON podem ser lidos diretamente pelo navegador usando o protocolo `file://`
- ☐ b) CORS é uma política que permite o acesso irrestrito a qualquer recurso na internet
- ☒ c) O navegador bloqueia a leitura de arquivos JSON locais (`file://`) por questões de segurança
- ☐ d) CORS só afeta requisições feitas com a função `fetch()` e não afeta arquivos JSON
- ☐ e) Para ler JSON localmente, basta renomear o arquivo para `.txt`

Questão 4 – Múltipla Escolha – 0,35

JavaScript

Considere o código JavaScript abaixo:

```
let a = 10;  
let b = "10";  
  
console.log(a == b);  
console.log(a === b);
```

Qual será a saída exibida no console?

- ☐ a) true e true
- ☒ b) true e false
- ☐ c) false e true
- ☐ d) false e false
- ☐ e) undefined e null

Questão 5 – Múltipla Escolha – 0,35

JavaScript

Analise o código abaixo que classifica a nota de um aluno:

```
let nota = 6;

if (nota >= 7) {
    console.log("Aprovado");
} else if (nota >= 5) {
    console.log("Recuperação");
} else {
    console.log("Reprovado");
}
```

Qual será a saída exibida no console?

- ☐ a) Aprovado
- ☒ b) Recuperação
- ☐ c) Reprovado
- ☐ d) Nenhuma saída
- ☐ e) Erro de sintaxe

Questão 6 – Múltipla Escolha – 0,35

JavaScript

Considere o array abaixo e as operações realizadas:

```
let frutas = ["Maçã", "Banana", "Laranja"];

frutas.push("Uva");
frutas.shift();
frutas.pop();
frutas.unshift("Morango");
```

Qual será o estado final do array `frutas` ?

- ☒ a) ["Morango", "Banana", "Laranja"]
- ☐ b) ["Morango", "Banana", "Uva"]
- ☐ c) ["Maçã", "Banana", "Morango"]
- ☐ d) ["Morango", "Banana", "Laranja", "Uva"]
- ☐ e) ["Banana", "Laranja", "Morango"]

Questão 7 – Questão Aberta – 0,45

Dissertativa

Explique detalhadamente como funciona o **OneCompiler** e qual a diferença entre ele e executar JavaScript diretamente no navegador.

Inclua em sua resposta: onde o código é executado, qual tecnologia é usada no servidor, como o resultado é exibido e por que muitos iniciantes se confundem sobre isso.

O navegador apenas exibe o resultado final. Diferença para execução no navegador:

Navegador: O código é interpretado e executado localmente no computador do usuário

Questão 8 – Questão Aberta – 0,45

Dissertativa

Explique detalhadamente o que é a política **CORS** e por que ela afeta a leitura de arquivos JSON.

Inclua em sua resposta: o significado de CORS, por que arquivos JSON não podem ser lidos localmente (file://) e quais ferramentas resolvem esse problema.

Cross-Origin Resource Sharing, é uma proteção do navegador. Quando abre file:// o java tenta usar o fetch("dados.json") o navegador pode bloquear a leitura. Para resolver é executar o projeto através de um servidor local HTTP

Questão 9 – Questão Aberta – 0,45

Dissertativa

Explique detalhadamente as diferenças entre **var** , **let** e **const** em JavaScript.

Inclua em sua resposta: as diferenças de escopo, reatribuição, redeclaração, exemplos práticos e qual é a boa prática recomendada atualmente.

Var→ Escopo de função. Evitar usar em código moderno.
Let → Escopo de bloco. Permite reatribuição.
const→ Escopo de bloco. Valor não pode ser alterado.

Questão 10 – Questão Aberta – 0,45

Dissertativa

Explique detalhadamente como funcionam os principais métodos de manipulação de arrays em JavaScript.

Inclua em sua resposta: o que fazem **push()** , **pop()** , **shift()** , **unshift()** e **splice()** , com exemplos práticos de uso.

push adiciona uma elemento no fim de um arrays. unshift adiciona um elemento ao inicio de arrays
pop remove um elemento ao fim de um arrays shift remove um elemento ao inicio de uma arays

**Gabarito Completo com Explicações**

Questão 1 – Verdadeiro (V) ou Falso (F)

✓ Resposta correta: **b) V – F – V**

Explicação detalhada:

- **I. VERDADEIRO** — O VS Code é gratuito, multiplataforma e suporta extensões como Live Server (servidor local com atualização automática) e Live Preview (visualização dentro do editor).
- **II. FALSO** — O GitHub Pages é **gratuito** para repositórios **públicos**. Na versão gratuita, não funciona com repositórios privados.
- **III. VERDADEIRO** — O OneCompiler **NÃO** executa código no navegador. Ele envia o código para servidores remotos que executam com Node.js e retornam o resultado.

Questão 2 – Verdadeiro (V) ou Falso (F)

✓ Resposta correta: **a) V – V – F**

Explicação detalhada:

- **I. VERDADEIRO** — `const` declara uma variável que **não pode ser reatribuída**. O valor é constante.
- **II. VERDADEIRO** — `let` tem **escopo de bloco** (respeita `{ }`) e **permite reatribuição** de valor.
- **III. FALSO** — `var` tem escopo de **função**, **NÃO** de bloco. Essa é uma das principais razões para evitar `var` em código moderno.

Boa prática: Use `const` por padrão, `let` quando precisar reatribuir, e **evite** `var`.

Questão 3 – Múltipla Escolha

✓ Resposta correta: **c) O navegador bloqueia a leitura de arquivos JSON locais (file://) por questões de segurança**

Explicação detalhada:

CORS (Cross-Origin Resource Sharing) é uma política de segurança dos navegadores que **impede que uma página web acesse recursos de outro domínio** sem permissão explícita.

- ✗ `file:///C:/site/index.html` → tentando ler `dados.json` → **CORS BLOQUEIA**
- ✓ `http://localhost:5500/index.html` (Live Server) → consegue ler JSON
- ✓ `https://usuario.github.io/site/` (GitHub Pages) → consegue ler JSON

Por que isso acontece? O navegador considera que arquivos locais (`file://`) não têm um servidor para autorizar o acesso, então bloqueia por segurança.

Soluções: Usar Live Server, GitHub Pages, OneCompiler ou qualquer servidor HTTP.

Questão 4 – Múltipla Escolha

✓ Resposta correta: **b) true e false**

Explicação detalhada:

Diferença entre == e ===:

- **== (igualdade)** — compara apenas o **valor**, convertendo tipos automaticamente.
- **=== (identidade)** — compara **valor E tipo**, sem conversão.

Analisando o código:

- `10 == "10" → true` (o valor é igual, JavaScript converte "10" para número)
- `10 === "10" → false` (tipos diferentes: number vs string)

💡 DICA: Use `===` sempre que possível para evitar conversões automáticas de tipo!

Questão 5 – Múltipla Escolha

✓ Resposta correta: **b) Recuperação**

Explicação detalhada:

Como funciona o if/else if/else:

- O programa avalia as condições em **ordem sequencial**
- Quando uma condição é **verdadeira**, executa aquele bloco e **sai** da estrutura

Analisando o código com nota = 6:

- 1º: `nota >= 7 → 6 >= 7 → FALSO`
- 2º: `nota >= 5 → 6 >= 5 → VERDADEIRO` → executa "Recuperação"
- O else não é executado porque uma condição já foi satisfeita

💡 DICA: A ordem das condições importa! Coloque as condições mais específicas primeiro.

Questão 6 – Múltipla Escolha

✓ Resposta correta: **a) ["Morango", "Banana", "Laranja"]**

Explicação detalhada:

Analisando passo a passo:

- **Inicial:** `["Maçã", "Banana", "Laranja"]`
- `push("Uva")` → adiciona ao final → `["Maçã", "Banana", "Laranja", "Uva"]`
- `shift()` → remove do início → `["Banana", "Laranja", "Uva"]`

- **pop()** → remove do final → ["Banana", "Laranja"]
- **unshift("Morango")** → adiciona ao início → ["Morango", "Banana", "Laranja"]

Resumo dos métodos:

- **push()** → adiciona ao **final**
- **pop()** → remove do **final**
- **unshift()** → adiciona ao **início**
- **shift()** → remove do **início**

Questão 7 – Questão Aberta **Sugestão de Resposta****Como funciona o OneCompiler:**

1. O usuário escreve o código no editor online
2. Ao executar, o código é **enviado para servidores remotos** da OneCompiler
3. O servidor executa o código usando **Node.js** (ambiente server-side)
4. O resultado da execução é enviado de volta ao navegador
5. O navegador **apenas exibe** o resultado final

Diferença para execução no navegador:

- **Navegador:** O código é interpretado e executado localmente no computador do usuário
- **OneCompiler:** O código é executado em servidores remotos, e apenas o resultado é mostrado

Por que isso confunde iniciantes:

- A interface parece similar a um console do navegador
- O resultado aparece rapidamente, dando a impressão de execução local
- Na verdade, é uma execução server-side que retorna apenas o output

Tecnologia usada: Node.js no servidor, que permite JavaScript rodar fora do navegador.

Questão 8 – Questão Aberta **Sugestão de Resposta****O que é CORS?**

CORS (Cross-Origin Resource Sharing) é uma **política de segurança dos navegadores** que impede que uma página web faça requisições para um domínio diferente do qual ela foi carregada, a menos que o servidor permita explicitamente.

Por que JSON local é bloqueado?

- Quando você abre um arquivo HTML diretamente (file:///C:/site/index.html), o navegador considera que a origem é o sistema de arquivos local
- Tentar ler dados.json do mesmo diretório é considerado uma requisição cross-origin
- O navegador **bloqueia** por segurança, pois não há um servidor para autorizar o acesso

Ferramentas que resolvem o problema:

- **Live Server (VS Code):** Cria um servidor HTTP local → requisições via HTTP, não file://
- **GitHub Pages:** Hospeda o site em um servidor HTTPS → requisições via protocolo HTTP/HTTPS
- **OneCompiler:** Já possui um servidor embutido que lida com as requisições corretamente

Como contornam:

- Servem os arquivos via **protocolo HTTP** em vez de file://
- O navegador trata requisições HTTP como **mesma origem** quando vem do mesmo servidor

Questão 9 – Questão Aberta

Sugestão de Resposta

1. var (antigo, evitar):

- Escopo de **função** (não respeita blocos { })
- Permite **reatribuição** e **redeclaração**
- Pode causar bugs difíceis de encontrar
- **Exemplo:** var nome = "João"; var nome = "Maria"; (funciona, mas é confuso)

2. let (moderno, para valores que mudam):

- Escopo de **bloco** { } (respeita if, for, etc.)
- Permite **reatribuição**, mas **NÃO permite redeclaração** no mesmo escopo
- **Exemplo:** let contador = 0; contador = 1;

3. const (recomendado para valores fixos):

- Escopo de **bloco** { }
- **NÃO permite reatribuição** nem redeclaração
- O valor deve ser definido na declaração
- **Exemplo:** const PI = 3.14159;

Boa prática atual:

-  Use **const** como padrão (para a maioria das variáveis)
-  Use **let** quando precisar reatribuir o valor

- **✗ Evite var** em código moderno (apenas para suporte a navegadores antigos)

Questão 10 – Questão Aberta

Sugestão de Resposta

1. push() - Adiciona ao final:

```
let frutas = ["Maçã", "Banana"];  
frutas.push("Laranja"); // ["Maçã", "Banana", "Laranja"]
```

2. pop() - Remove do final:

```
let ultima = frutas.pop(); // remove "Laranja" e retorna ela
```

3. unshift() - Adiciona ao início:

```
frutas.unshift("Morango"); // ["Morango", "Maçã", "Banana"]
```

4. shift() - Remove do início:

```
let primeira = frutas.shift(); // remove "Morango" e retorna  
ela
```

5. splice() - Remove/Substitui em posição específica:

```
// Remove 2 elementos a partir do índice 1  
frutas.splice(1, 2);  
  
// Substitui "Verde" por "Amarelo" no índice 1  
cores.splice(1, 1, "Amarelo");
```

Diferença entre os grupos:

- **push/pop:** Trabalham no **final** do array (mais eficientes)
- **unshift/shift:** Trabalham no **início** do array (menos eficientes, pois deslocam todos os elementos)
- **splice():** Trabalha em **qualquer posição** e é o mais versátil

Exemplo completo com splice:

```
let cores = ["Vermelho", "Verde", "Azul", "Amarelo"];  
cores.splice(1, 2, "Rosa", "Preto");  
// ["Vermelho", "Rosa", "Preto", "Amarelo"]  
// Removeu "Verde" e "Azul", adicionou "Rosa" e "Preto"
```